

Java Full Stack Development

3. Maven Concepts



Module Topics

- Maven Introduction
- The POM and Project Structure
- Fat Jars
- IntelliJ Integration

Maven Introduction

Maven

- Build automation and project management tool for Java projects
 - Manages the entire build lifecycle
 - Compilation, testing, packaging, and deployment
 - Provides a standardized project structure adopted across the Java ecosystem
 - Acts as a dependency manager
 - Automatically downloads and wires together the libraries a project needs
- Driven by a single configuration file
 - The POM: Project Object Model
- Understanding Maven works is foundational to the material that follows



Maven

- Created at Apache in 2002 to solve two specific problems
 - Inconsistent project structures across teams
 - Painful manual management of JAR files and their transitive dependencies
 - Before Maven
 - Developers would download JARs, place them in project folders, and manually track version compatibility
 - Maven introduced a declarative, convention-driven model
 - The most widely used Java build tool in enterprise environments,



Maven

- Design goals: problems to be solved
 - Problem 1: Dependency hell
 - Java projects depend on dozens (often hundreds) of external libraries
 - Libraries themselves have their own dependencies so the dependency graph grows quickly
 - Manual management leads to version conflicts and missing JARs at runtime
 - Problem 2: Inconsistent project layouts
 - Without a standard, every team structures projects differently
 - Onboarding onto a new project required learning a custom build process
 - Problem 3: Repeatable builds
 - Builds that work on one machine but fail on another due to environment differences



Maven

- Maven provides
 - Central repository for all Java artifacts
 - Maven's central repository (Maven Central) currently hosts over 500,000 artifacts
 - Declarative dependency resolution with transitive dependency management
 - Maven downloads not just a library, but everything that library needs, resolving the entire dependency graph automatically
 - Convention over configuration
 - One standard project layout for all
 - if you follow Maven's standard directory layout, almost no custom build scripting is required



The POM and Project Structure

The POM

- A minimal POM is shown
 - **groupId**: Reverse-domain namespace
 - Like a package name for the project
 - **artifactId**: The project/module name
 - **version**: Semantic version
 - SNAPSHOT means under active development
 - The **groupId**, **artifactId** and **version** together form a globally unique identifier for any artifact in the Java ecosystem
 - Known as Maven Coordinates or GAV coordinates
 - These same coordinates are used when other projects declare a dependency this project

```
<project>
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>my-service</artifactId>
  <version>1.0.0-SNAPSHOT</version>
  <packaging>jar</packaging>

</project>
```

Standard Directory Layout

- This layout is a universal standard across the Java ecosystem
 - Maven automatically knows where to find sources, resources, and tests
 - IDEs (IntelliJ IDEA, Eclipse, VS Code) recognise this layout natively
 - Spring Boot projects follow this layout exactly
 - When Spring Initializr generates a new Spring Boot project, it uses this exact layout

```
my-service/  
├─ pom.xml  
└─ src/  
    ├─ main/  
    │   ├─ java/           ← Application source code  
    │   └─ resources/      ← Config files, templates, static assets  
    └─ test/  
        ├─ java/           ← Test source code  
        └─ resources/      ← Test-specific config files
```

Dependency Declaration

- Scope is a critical concept
 - Test scope dependencies are not bundled into your final artifact
 - They exist only during test compilation and execution
 - This keeps production artifacts lean
 - Provided scope is used when the runtime container supplies the dependency
 - The code needs it to compile, but should not package it.
 - Spring Boot development primarily works with compile and test.

```
<dependencies>

<!-- Spring Boot Web Starter -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
  <version>3.2.0</version>
</dependency>

<!-- JUnit 5 – test scope only -->
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter</artifactId>
  <version>5.10.0</version>
  <scope>test</scope>
</dependency>

</dependencies>
```

Dependency Declaration

Dependency Scopes:

Scope	Available At	Included In JAR?
<code>compile</code> (default)	All phases	Yes
<code>test</code>	Test compile & run only	No
<code>runtime</code>	Runtime only	Yes
<code>provided</code>	Compile only (container provides at runtime)	No

```
<dependencies>

<!-- Spring Boot Web Starter -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
  <version>3.2.0</version>
</dependency>

<!-- JUnit 5 – test scope only -->
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter</artifactId>
  <version>5.10.0</version>
  <scope>test</scope>
</dependency>

</dependencies>
```

Transitive Dependencies

- When you declare dependency A
 - And A depends on B and C
 - Maven automatically resolves B and C
 - This is called transitive dependency resolution
- The full dependency tree can be inspected with the command
 - `mvn dependency:tree`
 - Sample output shown

```
com.example:my-service:jar:1.0.0-SNAPSHOT
├─ org.springframework.boot:spring-boot-starter-web:jar:3.2.0
│   ├─ org.springframework.boot:spring-boot-starter:jar:3.2.0
│   │   ├─ org.springframework:spring-core:jar:6.1.0
│   │   └─ org.springframework:spring-context:jar:6.1.0
│   └─ org.springframework:spring-webmvc:jar:6.1.0
└─ com.fasterxml.jackson.core:jackson-databind:jar:2.16.0
```

Transitive Dependencies

- If two paths require different versions of the same library
 - Maven uses nearest-wins
 - The version closest to the root of the dependency tree wins
 - Maven provides [dependency:analyze](#)
 - Identifies declared but unused dependencies
 - Identifies used but undeclared dependencies
 - Useful for keeping the POM clean
 - In large microservice projects, keeping dependencies intentional and minimal is critical

```
com.example:my-service:jar:1.0.0-SNAPSHOT
└─ org.springframework.boot:spring-boot-starter-web:jar:3.2.0
   │ └─ org.springframework.boot:spring-boot-starter:jar:3.2.0
   │   │ └─ org.springframework:spring-core:jar:6.1.0
   │   │   └─ org.springframework:spring-context:jar:6.1.0
   │   └─ org.springframework:spring-webmvc:jar:6.1.0
   └─ com.fasterxml.jackson.core:jackson-databind:jar:2.16.0
```

The Build Lifecycle

- Maven defines three built-in lifecycles, each composed of ordered phases
 - The default lifecycle is the most common
 - Shown on the right
 - When a phase is invoked, Maven executes all preceding phases in order
 - `mvn package`
 - runs: validate → compile → test → package
 - `mvn install`
 - runs: validate → compile → test → package → verify → install

Phase	What Happens
<code>validate</code>	Validate the project structure
<code>compile</code>	Compile source code
<code>test</code>	Run unit tests
<code>package</code>	Package compiled code (e.g., into a JAR)
<code>verify</code>	Run integration tests and checks
<code>install</code>	Install the artifact into the local repository
<code>deploy</code>	Deploy the artifact to a remote repository

Core Commands

- The `target/` directory holds all build output
 - Compiled classes, test results, and packaged artifacts
- `mvn clean package`
 - Most common command in Java dev
 - The `clean` phase deletes the `target/` directory to ensure a fresh build
 - `spring-boot:run` goal is useful during development
 - It launches the Spring Boot application directly through Maven without needing to first package it

```
# Compile source code
```

```
mvn compile
```

```
# Run unit tests
```

```
mvn test
```

```
# Package into a JAR or WAR
```

```
mvn package
```

```
# Install to local repository (~/.m2)
```

```
mvn install
```

```
# Clean the target/ directory (remove previous build output)
```

```
mvn clean
```

```
# Common combination: clean then package
```

```
mvn clean package
```

```
# Skip tests during build (use sparingly)
```

```
mvn clean package -DskipTests
```

```
# Run a specific plugin goal directly
```

```
mvn dependency:tree
```

```
mvn spring-boot:run
```

Maven Plugins

- Maven's build lifecycle is implemented entirely through plugins
 - Each phase binds to one or more plugin goals
 - Plugins are themselves Maven artifacts
 - Declared in the POM and downloaded automatically

Phase	Default Plugin	Goal
compile	maven-compiler-plugin	compile
test	maven-surefire-plugin	test
package	maven-jar-plugin	jar

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.12.0</version>
      <configuration>
        <source>21</source>
        <target>21</target>
      </configuration>
    </plugin>
  </plugins>
</build>
```

Maven Plugins

- Everything Maven does is implemented as a plugin
 - Compiling, testing, packaging, generating documentation, running code quality checks, deploying
 - The Spring Boot Maven plugin
 - Adds goals like `spring-boot:run` and `spring-boot:build-image`
 - Plugins are configured declaratively in the `<build>` section of the POM.

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.12.0</version>
      <configuration>
        <source>21</source>
        <target>21</target>
      </configuration>
    </plugin>
  </plugins>
</build>
```

Spring Boot Parent POM

- Spring Boot projects inherit from a parent POM
 - Pre-configures almost everything
 - Pre-configured compiler settings (Java version)
 - Managed versions for hundreds of compatible dependencies
 - Default plugin configurations (Surefire, compiler, etc.)
 - Resource filtering configuration
 - Rarely need to specify versions for Spring dependencies

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.2.0</version>
</parent>
```

Spring Boot Parent POM

- The Spring Boot parent POM
 - Encapsulates a tested, compatible set of dependency versions in what is called a Bill of Materials (BOM)
 - You can still override individual versions if necessary, but in practice this is rarely needed
 - Significant productivity boost
 - Version conflicts, largely disappear when working within the Spring Boot ecosystem.

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
    <!-- No version required – inherited from parent BOM -->
  </dependency>
</dependencies>
```

Spring Boot Maven Plugin

- The `spring-boot-maven-plugin`
- Central to running and packaging Spring Boot applications

```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
```

Goal	Purpose
<code>spring-boot:run</code>	Run the application during development
<code>spring-boot:repackage</code>	Create an executable "fat JAR" (runs via <code>java -jar</code>)
<code>spring-boot:build-image</code>	Build an OCI container image without a Dockerfile

Local Repository

- `~/.m2/repository`:
 - The local cache on your development machine
- Maven Central
 - The public central repository
- Internal repositories
 - Enterprise repositories that host private and proxied artifacts
- Enterprise pattern
 - All dependency requests route through an internal repository instance
 - Provides security scanning, license management, and artifact caching

```
Your Project → pom.xml declares dependency
↓
Maven checks ~/.m2/repository (local cache)
↓ (if not found)
Maven downloads from Maven Central (or configured remote repo)
↓
Artifact cached in ~/.m2/repository
↓
Artifact available to your project
```

Enterprise Configuration

- The `~/.m2/settings.xml` file configures Maven at the user level
 - For environment-specific and sensitive configuration
 - Separate from the project's `pom.xml` which is committed to source control
 - Credentials should never be hardcoded
 - Use environment variable references as shown
 - In many enterprise setups, a pre-configured `settings.xml` is used by all developers
 - Ensures everyone uses the same repository mirrors and no manual setup is required
 - The `pom.xml` describes the project
 - While `settings.xml` describes the environment. Keeping this separation clean is important

```
<settings>
  <mirrors>
    <!-- Route all requests through internal Nexus -->
    <mirror>
      <id>nexus</id>
      <mirrorOf>*</mirrorOf>
      <url>https://nexus.internal.example.com/repository/maven-public/</url>
    </mirror>
  </mirrors>

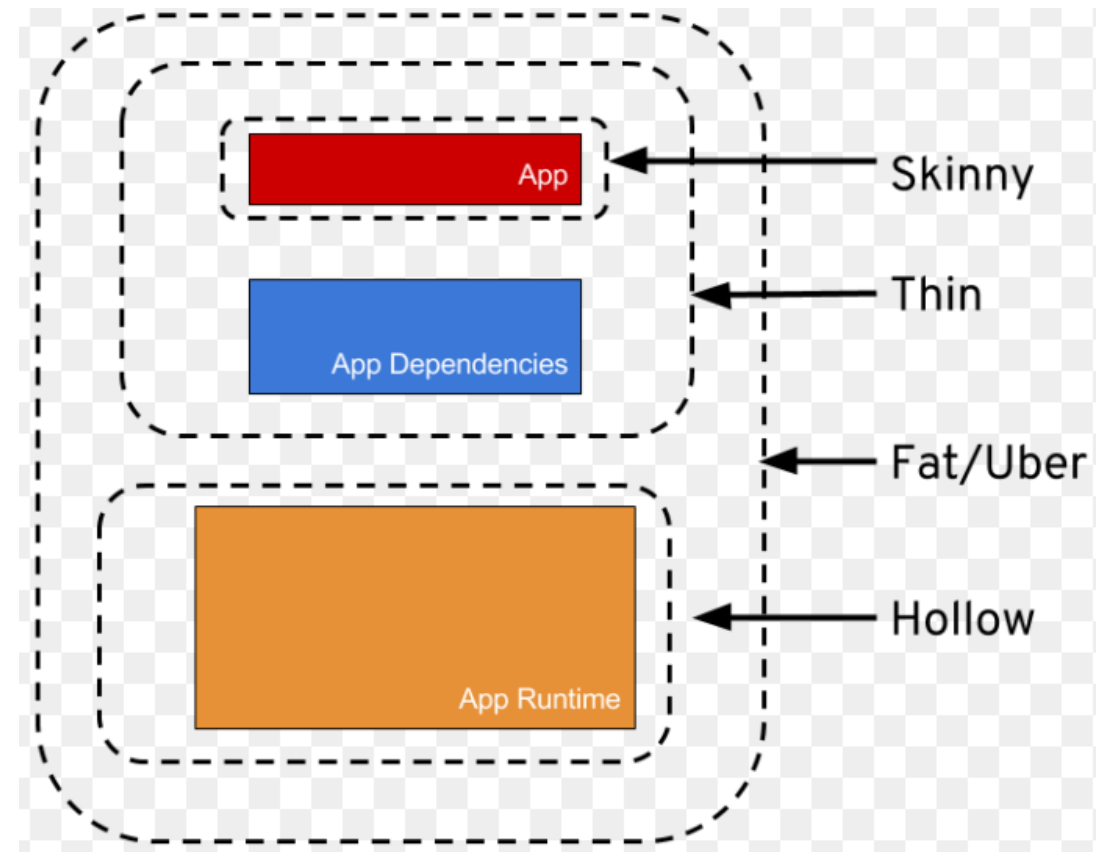
  <servers>
    <!-- Credentials for authenticated repositories -->
    <server>
      <id>nexus</id>
      <username>${env.NEXUS_USER}</username>
      <password>${env.NEXUS_PASSWORD}</password>
    </server>
  </servers>

  <profiles>
    <!-- Environment-specific configuration -->
  </profiles>
</settings>
```

Fat Jars

Executable Fat JAR

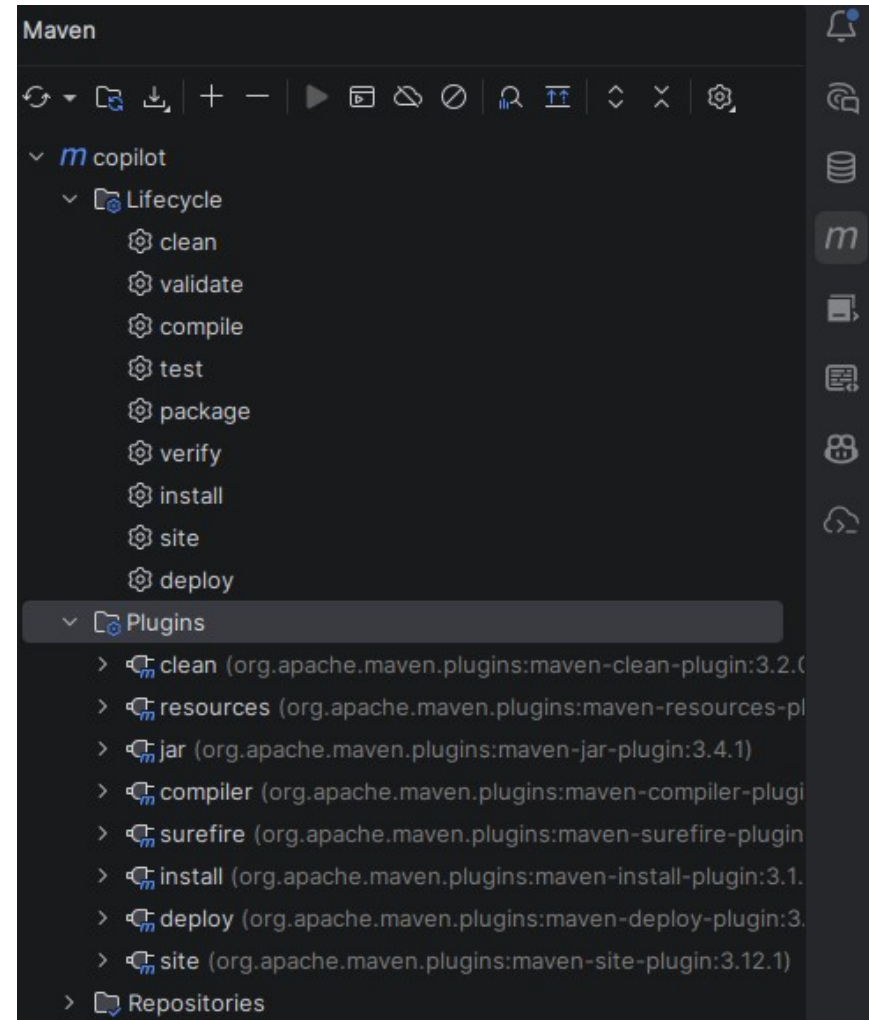
- The Executable Fat JAR
 - Packages the application AND all its dependencies into a single self-contained jar file
 - Deployed and run with
 - `java -jar my-service-1.0.0.jar`
 - Standard deployment unit for Spring Boot microservices
 - This model maps perfectly to containerized deployment
 - The fat JAR goes into a Docker/OCI image and runs as a microservice



IntelliJ Integration

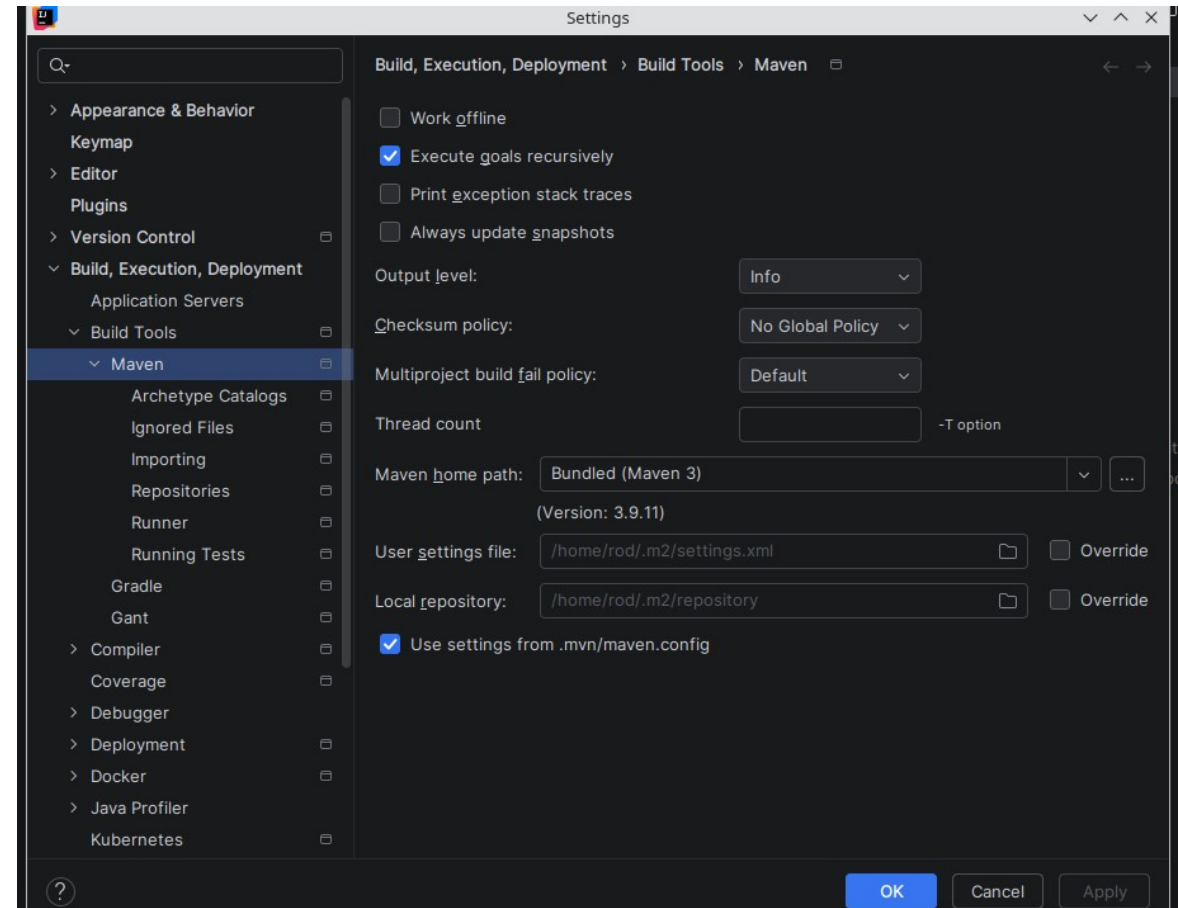
Maven Tool Window

- Graphical interface to the Maven build model
 - Key panels within the window:
 - Lifecycle: the standard Maven lifecycle phases; double-click any phase to run it
 - Plugins: all plugins declared in the [pom.xml](#), with their individual goals exposed
 - Dependencies: a navigable tree of all resolved dependencies
 - The toolbar provides:
 - Reload (circular arrow) — re-imports the project model from pom.xml
 - Execute Maven Goal (play button with m) — runs an arbitrary goal string
 - Toggle Skip Tests: adds [-DskipTests](#) to all Maven executions from this window



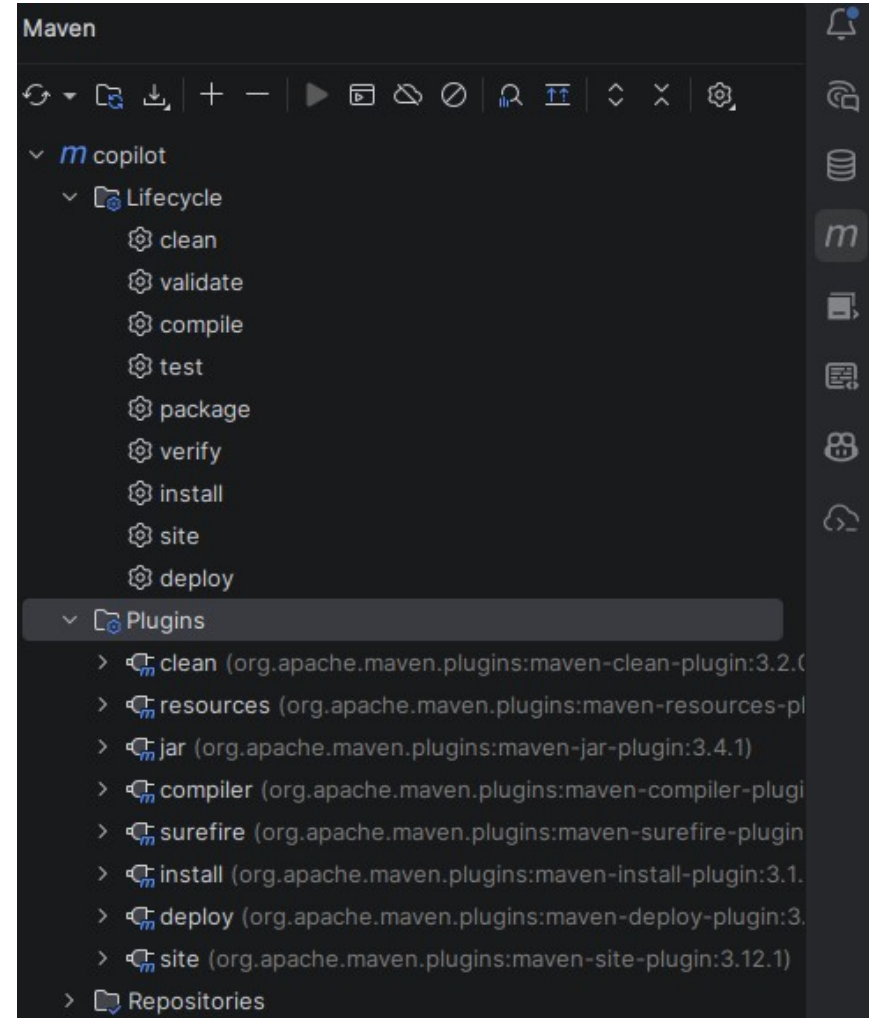
Maven Tool Settings

- Allows customization of Maven behavior
 - Note that we can specify the remote repositories to be used
 - This ensures we are only using trusted libraries



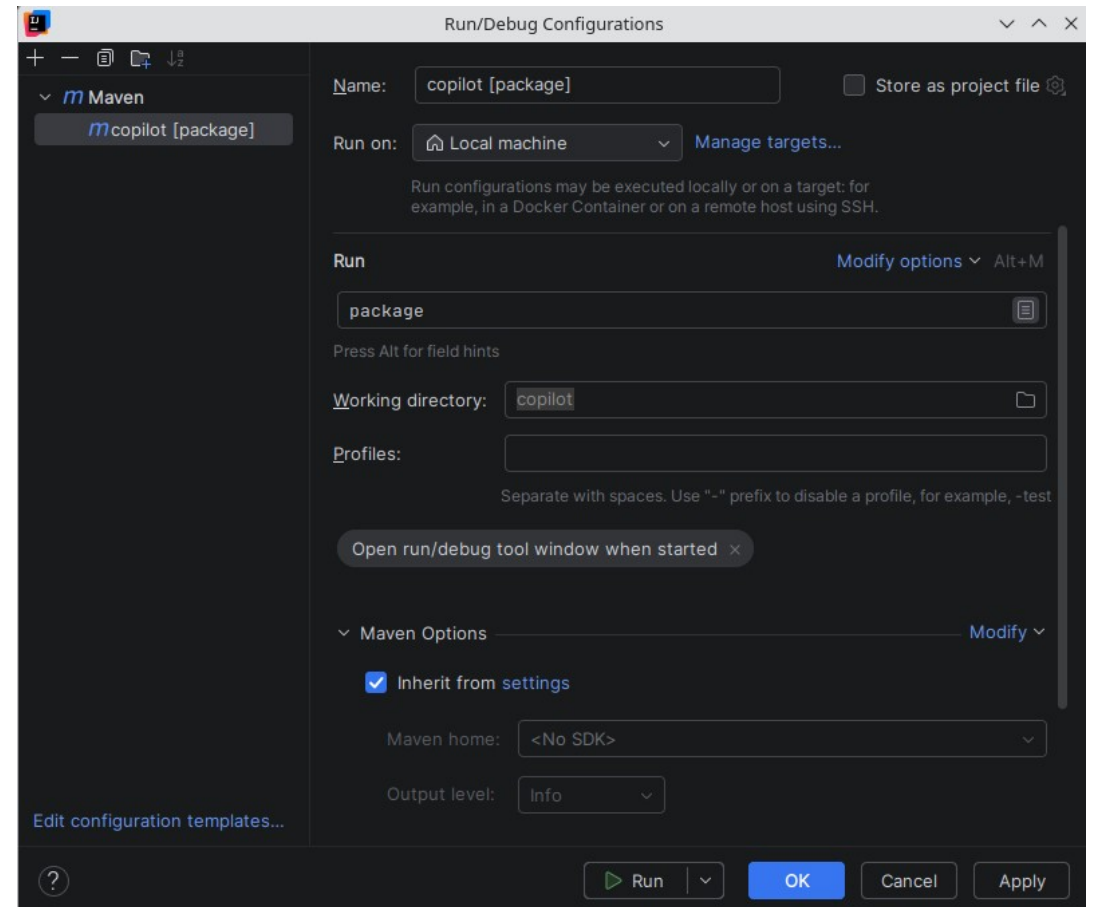
Running Maven Goals

- Double-click in the Maven Tool Window Lifecycle panel
 - The fastest way to run standard lifecycle phases
 - Double-click test to compile and run all unit tests
 - Double-click verify to run the full build including integration tests
- Execute Maven Goal dialog
 - Toolbar icon in the Maven Tool Window (play button with m)
 - Or: **Ctrl+Shift+Alt+R**
 - Accepts any valid Maven goal string: **clean verify**
 - Supports command-line flags: **-DskipTests, -pl module-name, -am**



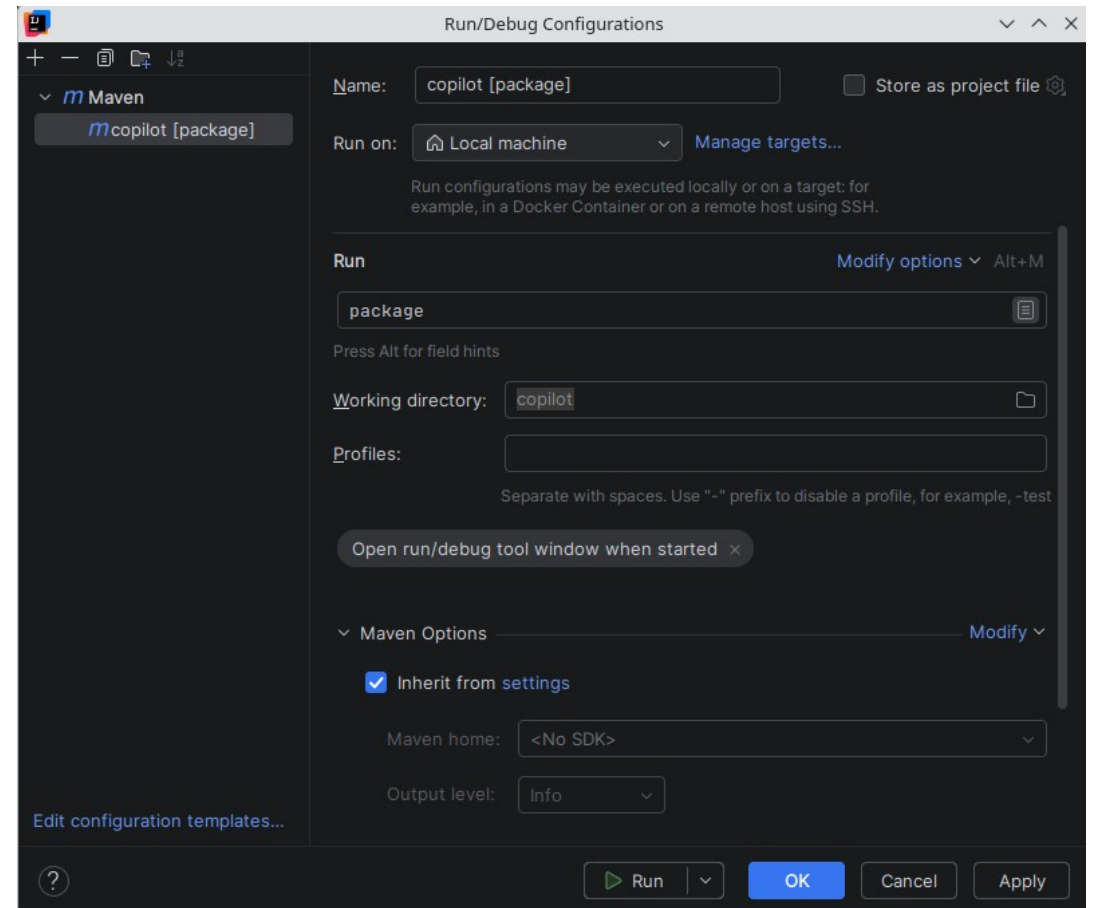
Running Maven Goals

- Maven Run Configuration
 - Saves a Maven goal string as a named, reusable execution
 - Created via
 - [Run](#) → [Edit Configurations](#) → [+](#) → [Maven](#)
- Key fields
 - Working directory
 - The project or module root where Maven run
 - Command line
 - [The full goal string, e.g.](#) clean verify -Plocal



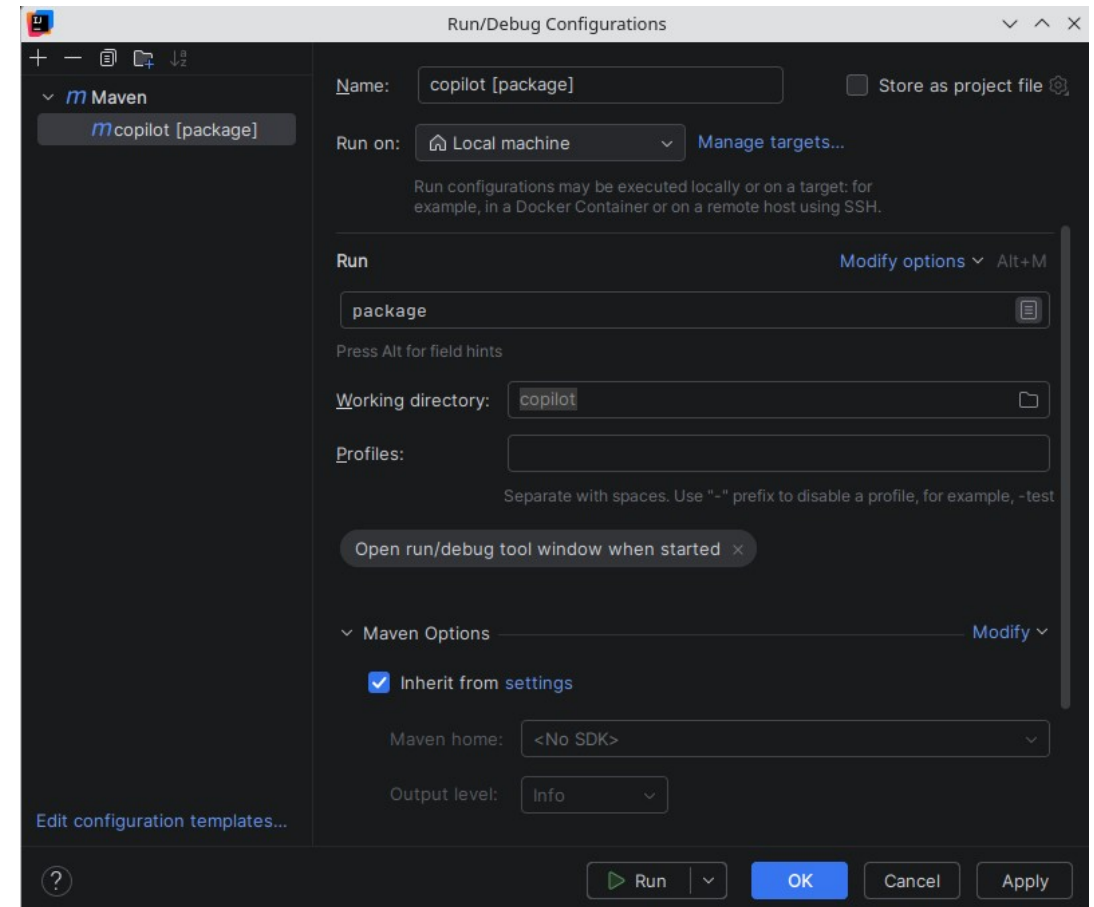
Running Maven Goals

- Key fields
 - Profiles
 - Maven profiles to activate (alternative to -P in the command line)
 - Properties
 - Maven properties to set (alternative to -D in the command line)
 - JRE
 - Which JVM to use for the Maven process itself
 - Can be saved as Shared
 - Vommitted to [.idea/runConfigurations/](#) for team-wide use



Running Tests

- Maven runs tests through two distinct plugins:
 - Surefire plugin: unit tests, runs during the test phase
 - Failsafe plugin: integration tests, runs during the verify phase
- Surefire picks up test classes by naming convention
 - `**/*Test.java`, `**/*Tests.java`, `**/*TestCase.java`
- Failsafe picks up integration test classes by naming convention
 - `**/*IT.java`, `**/*ITCase.java`, `**/*IntegrationTest.java`
- Test reports are written to:=
 - Unit tests: `target/surefire-reports/`
 - Integration tests: `target/failsafe-reports/`



Running Tests

- Maven runs tests through two distinct plugins:
 - Surefire plugin: unit tests, runs during the test phase
 - Failsafe plugin: integration tests, runs during the verify phase
- Surefire picks up test classes by naming convention
 - `**/*Test.java`, `**/*Tests.java`, `**/*TestCase.java`
- Failsafe picks up integration test classes by naming convention
 - `**/*IT.java`, `**/*ITCase.java`, `**/*IntegrationTest.java`
- Test reports are written to
 - Unit tests: `target/surefire-reports/`
 - Integration tests: `target/failsafe-reports/`

```
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.morningstar.planadvantage:aws-service-utilities >-----
[INFO] Building aws-service-utilities 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- maven-failsafe-plugin:2.22.0:integration-test (default-cli) @ aws-service-utilities ---
[INFO] Failsafe report directory: /Users/afreymi/Desktop/repos/plan-advantage-api/target/failsafe-reports
[INFO]
[INFO] -----<!-- Dependencies omitted. pretty sure they are there -->-----
[INFO]
[INFO] T E S T S
[INFO]
[INFO] -----<build>-----
[INFO]
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 0, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----<executions>-----
[INFO] BUILD SUCCESS
[INFO]
[INFO] -----<execution>-----
[INFO]
[INFO] -----<goals>-----
[INFO] Total time: 2.112 s
[INFO] Finished at: 2019-01-22T11:32:53-06:00
[INFO]
[INFO] ----->[INFO]-----
```

Running Tests

- The naming convention for test classes is not arbitrary
 - Surefire and Failsafe plugins use file naming patterns to decide which classes to run and when
 - Name integration tests with the **IT** suffix or **IntegrationTest** suffix
 - Determines which Maven lifecycle phase picks them up
 - A class `OrderServiceTest.java` runs during `mvn test`
 - A class `OrderServiceIT.java` runs during `mvn verify`

```
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.morningstar.planadvantage:aws-service-utilities >-----
[INFO] Building aws-service-utilities 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- maven-failsafe-plugin:2.22.0:integration-test (default-cli) @ aws-service-utilities ---
[INFO] Failsafe report directory: /Users/afreymi/Desktop/repos/plan-advantage-api/sr
[INFO]
[INFO] -----<!-- Dependencies omitted. pretty sure they are there -->-----
[INFO]
[INFO] T E S T S
[INFO]
[INFO] -----<build>-----
[INFO]
[INFO] <plugins>
[INFO] Results:
[INFO]   <plugin>
[INFO]     <artifactId>maven-failsafe-pl
[INFO] Tests run: 0, Failures: 0, Errors: 0, Skipped: 0
[INFO]     <version>2.22.0</version>
[INFO]   </plugin>
[INFO] </plugins>
[INFO] BUILD SUCCESS
[INFO]
[INFO] -----<goals>-----
[INFO] Total time: 2.112 s
[INFO] Finished at: 2019-01-22T11:32:53-06:00
[INFO]
[INFO] -----<goal>integration-test</goal>
[INFO] -----<goal>verify</goal>-----
```

Running Tests

- IntelliJ provides two ways to run tests
 - IntelliJ's native JUnit runner
 - Default when clicking the gutter icon or [Ctrl+Shift+F10](#)
 - Maven's Surefire runner
 - Used when running [mvn test](#) or [mvn verify](#)

	IntelliJ Native Runner	Maven Surefire Runner
Speed	Faster — no Maven startup overhead	Slower — Maven initializes first
Test UI	Rich — tree view, diff on failure, coverage	Console output only
Classpath	Managed by IntelliJ's project model	Managed by Maven's dependency resolution
CI Fidelity	May differ from CI behaviour	Matches the CI pipeline exactly
Best used for	Daily development cycle	Pre-push verification, CI debugging

Key Shortcuts

Action	Method
Open Maven Tool Window	View → Tool Windows → Maven
Reload Maven project	Reload icon in Maven Tool Window toolbar
Run a lifecycle phase	Double-click phase in Maven Tool Window
Execute arbitrary Maven goal	Toolbar play icon () in Maven Tool Window
Execute Maven goal via keyboard	Ctrl+Shift+Alt+R
Run <code>mvn test</code> for a single class (delegated)	Gutter icon → Ctrl+Shift+F10
Open Run Configurations	Alt+Shift+F10
Toggle skip tests	Toggle button in Maven Tool Window toolbar

Questions?

